

POLITECNICO
MILANO 1863

Hoxha Eldi, Prunescu Bogdan-Andrei,
Marian Sorin-Nicu

TAVI: Trajectory-Level Verification of Embodied AI Agents via LTL

Deliverable:	Brief Scientific Report
Title:	TAVI: Trajectory-Level Verification of Embodied AI Agents via LTL
Authors:	Hoxha Eldi, Prunescu Bogdan-Andrei, Marian Sorin-Nicu
Supervisor:	Matteo Camilli
Version:	1
Date:	

Contents

Table of Contents **3**

List of Figures **4**

List of Tables **4**

1 Introduction **5**

2 Related Work **5**

3 Method **5**
3.1 Pipeline Overview 5
3.2 Trace Collection and Predicate Vocabulary 5
3.3 Rule Generation 5
3.4 LTL Checking 5
3.5 Implementation Notes 6

4 Evaluation Setup **6**

5 Results **6**

6 Discussion and Limitations **6**

7 Conclusion and Next Steps **6**

8 References **6**

List of Figures

List of Tables

1 Introduction

Motivate the problem: per-step checking (AgentSpec) misses hazards that only show up across a trajectory (e.g. an object made unsafe by an earlier action). State in 2-3 sentences what TAVI does differently (temporal, auto-generated LTL rules over a closed predicate vocabulary, runtime monitor). End with a one-sentence roadmap of the rest of the report.

2 Related Work

One short paragraph each on: (1) AgentSpec – trigger/check/enforce DSL, per-step only, manually authored rules; (2) SafeAgentBench – safety-aware task planning benchmark, evaluates agent behavior, not a checking/monitoring mechanism itself, this is our evaluation benchmark (Section 4); (3) Embodied AI Agents (Fung et al.) – virtual/wearable/-physical taxonomy, world-model framing, justifies our physical-agent-only scope; (4) Specification Patterns for Robotic Missions – the 22 LTL/CTL mission patterns our rule templates are instantiated from (Section 3.3). Close with 2-3 sentences stating precisely what TAVI adds relative to all four: trajectory-level temporal checking, automatic rule generation, closed-vocabulary grounding.

3 Method

3.1 Pipeline Overview

Describe the 5 conceptual modules (trace collector, rule generation, trace translation/grounding, LTL checking, runtime monitor); state which are implemented end-to-end vs. still placeholder (violation reporting/NL explanation is not implemented yet). Consider a simple pipeline diagram (Figure) showing data flow between modules.

3.2 Trace Collection and Predicate Vocabulary

Describe the JSONL trace schema (step id, timestamp, action, pre/post state, observation, propositions, llm_call, tool_calls, outcome). Describe the grounded predicate vocabulary extracted from AI2-THOR metadata (visible, near, hot/cold, material, filled_with, capability flags, semantic hazard categories). Explain the closed-vocabulary contract (trace_predicates.md) and the failure mode it fixed: LLM-generated rules referencing predicates absent from the trace, which vacuously passed.

3.3 Rule Generation

Describe online, per-task, cached LTL rule generation via Gemini 2.5 Flash; the two-stage process (pattern selection from the specification-patterns catalogue, then grounding with object IDs observed in the scene); why caching keeps LLM cost constant regardless of trajectory length.

3.4 LTL Checking

Describe flloat/LTLf checking mechanics, what "violation" means operationally over a finite trace, and how the runtime monitor (main.py) evaluates rules incrementally and halts the agent mid-task on violation rather than only reporting after the fact.

3.5 Implementation Notes

Pick 2-3 concrete engineering details worth reporting: the agent_executed detection fix (planner error strings no longer counted as executed actions); the parallel benchmark runner (run_phases.py, 4 files concurrently); the Gemini Flash vs Pro tradeoff on long-horizon tasks (SR 0.86 vs 0.61 with 16k token budget) – note this was found in earlier benchmarking outside the current pipeline.

4 Evaluation Setup

Describe SafeAgentBench (750 tasks, 10 hazard types), the 4 task categories used (unsafe/safe/abstract/long-horizon) with 1-line definitions, sample size (100 tasks, 25 per category), and the outcome taxonomy (TP/TN/FN/FP/Agent Avoided Action/No Trace) with 1-line definitions of each as a table. Mention the two project-level constraints from the supervisor meeting that shape this section: physical-agent-only scope, and benchmark-driven (not analytically modeled) confidence.

5 Results

Insert the 100-task results table (TP 23/39, TN 50/50, FN 16, FP 0, Agent Avoided Action 2, No Trace 9) and headline metrics (82% accuracy, 59% recall, 100% precision). Include the comparison table against AgentSpec (71% recall / 96% precision, no temporal reasoning). Also report the earlier 40-task run (92.5% accuracy) alongside the 100-task run, since it sets up the overfitting discussion in Section 6.

6 Discussion and Limitations

Cover: overfitting as task diversity increases (92.5% on 40 tasks to 82% on 100 tasks); the recall bottleneck of 16 missed unsafe tasks despite 100% precision, worst on the abstract category where the hazard is described indirectly; the 9 no-trace tasks being agent/planner crashes upstream of the checker (not checker failures) that still reduce effective coverage; confidence intervals must be benchmark-driven per supervisor constraint and are not yet exposed (pipeline currently reports a boolean satisfied/violated verdict only); and the scope being limited to physical embodied agents per project constraints.

7 Conclusion and Next Steps

One paragraph summarizing the contribution and headline results, then concrete next steps: diagnose specific abstract/long-horizon failure cases (vocabulary too narrow vs. LLM under-specifying the formula); condition rule generation on hazard type (SafeAgentBench's 10 categories) instead of free-form instruction text, to reduce overfitting; implement violation reporting with natural-language explanation/highlighting; investigate and consistently report the 9 no-trace tasks.

8 References

Papers in Docs/Papers/ – cite these first. Fill in full bibliographic details (authors, venue, year, pages) for each before submission.

e, year

1. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents.

2. SafeAgentBench: a benchmark for safety-aware task planning of embodied LLM agents.
3. Embodied AI Agents: Modeling the World.
4. Specification Patterns for Robotic Missions.

Other sources researched during the project (tools, libraries, frameworks) – add as used, with full citation/URL for each we actually relied on.

- AI2-THOR simulation environment.
- LangGraph agent framework.
- flloat LTLf checking library.
- ProbGuard – LangChain-based probabilistic runtime monitoring.
- Gemini 2.5 Flash / Pro model documentation.

Todo list

Motivate the problem: per-step checking (AgentSpec) misses hazards that only show up across a trajectory (e.g. an object made unsafe by an earlier action). State in 2-3 sentences what TAVI does differently (temporal, auto-generated LTL rules over a closed predicate vocabulary, runtime monitor). End with a one-sentence roadmap of the rest of the report. 5

One short paragraph each on: (1) AgentSpec – trigger/check/enforce DSL, per-step only, manually authored rules; (2) SafeAgentBench – safety-aware task planning benchmark, evaluates agent behavior, not a checking/monitoring mechanism itself, this is our evaluation benchmark (Section 4); (3) Embodied AI Agents (Fung et al.) – virtual/wearable/physical taxonomy, world-model framing, justifies our physical-agent-only scope; (4) Specification Patterns for Robotic Missions – the 22 LTL/CTL mission patterns our rule templates are instantiated from (Section 3.3). Close with 2-3 sentences stating precisely what TAVI adds relative to all four: trajectory-level temporal checking, automatic rule generation, closed-vocabulary grounding. 5

Describe the 5 conceptual modules (trace collector, rule generation, trace translation/grounding, LTL checking, runtime monitor); state which are implemented end-to-end vs. still placeholder (violation reporting/NL explanation is not implemented yet). Consider a simple pipeline diagram (Figure) showing data flow between modules. 5

Describe the JSONL trace schema (step id, timestamp, action, pre/post state, observation, propositions, llm_call, tool_calls, outcome). Describe the grounded predicate vocabulary extracted from AI2-THOR metadata (visible, near, hot/cold, material, filled_with, capability flags, semantic hazard categories). Explain the closed-vocabulary contract (trace_predicates.md) and the failure mode it fixed: LLM-generated rules referencing predicates absent from the trace, which vacuously passed. 5

Describe online, per-task, cached LTL rule generation via Gemini 2.5 Flash; the two-stage process (pattern selection from the specification-patterns catalogue, then grounding with object IDs observed in the scene); why caching keeps LLM cost constant regardless of trajectory length. 5

Describe flloat/LTLf checking mechanics, what "violation" means operationally over a finite trace, and how the runtime monitor (main.py) evaluates rules incrementally and halts the agent mid-task on violation rather than only reporting after the fact. 5

Pick 2-3 concrete engineering details worth reporting: the agent_executed detection fix (planner error strings no longer counted as executed actions); the parallel benchmark runner (run_phases.py, 4 files concurrently); the Gemini Flash vs Pro tradeoff on long-horizon tasks (SR 0.86 vs 0.61 with 16k token budget) – note this was found in earlier benchmarking outside the current pipeline. 6
Describe SafeAgentBench (750 tasks, 10 hazard types), the 4 task categories used (unsafe/safe/abstract/long-horizon) with 1-line definitions, sample size (100 tasks, 25 per category), and the outcome taxonomy (TP/TN/FN/FP/Agent Avoided Action/No Trace) with 1-line definitions of each as a table. Mention the two project-level constraints from the supervisor meeting that shape this section: physical-agent-only scope, and benchmark-driven (not analytically modeled) confidence. 6
Insert the 100-task results table (TP 23/39, TN 50/50, FN 16, FP 0, Agent Avoided Action 2, No Trace 9) and headline metrics (82% accuracy, 59% recall, 100% precision). Include the comparison table against AgentSpec (71% recall / 96% precision, no temporal reasoning). Also report the earlier 40-task run (92.5% accuracy) alongside the 100-task run, since it sets up the overfitting discussion in Section 6. 6
Cover: overfitting as task diversity increases (92.5% on 40 tasks to 82% on 100 tasks); the recall bottleneck of 16 missed unsafe tasks despite 100% precision, worst on the abstract category where the hazard is described indirectly; the 9 no-trace tasks being agent/planner crashes upstream of the checker (not checker failures) that still reduce effective coverage; confidence intervals must be benchmark-driven per supervisor constraint and are not yet exposed (pipeline currently reports a boolean satisfied/violated verdict only); and the scope being limited to physical embodied agents per project constraints. 6
One paragraph summarizing the contribution and headline results, then concrete next steps: diagnose specific abstract/long-horizon failure cases (vocabulary too narrow vs. LLM underspecifying the formula); condition rule generation on hazard type (SafeAgentBench's 10 categories) instead of free-form instruction text, to reduce overfitting; implement violation reporting with natural-language explanation/highlighting; investigate and consistently report the 9 no-trace tasks. 6
Papers in Docs/Papers/ – cite these first. Fill in full bibliographic details (authors, venue, year, pages) for each before submission. 6
authors, venue, year 6
authors, venue, year 7
authors (Fung et al.), venue, year 7
authors, venue, year 7
Other sources researched during the project (tools, libraries, frameworks) – add as used, with full citation/URL for each we actually relied on. 7
paper/URL 7
docs/URL 7
paper/URL 7
mentioned 2026-04-01 meeting; confirm if actually used or drop 7
URL 7